

C146 final project — Spring 2026

Predicting the impact of genetic variants in ARSA

Data Science for Biology — PMB / MCB / BioE / CompBio C146

Due: Friday 1 May 2026, 10:00 pm, on Gradescope.

This is a living document, v12b. *Details will be refined over the next two weeks — in particular the exact list of ARSA variants, the release timing of files, and leaderboard mechanics. Expect updates; check the course site.*

This is the final project of C146, and it is meant to be a culmination of what you have learned this semester: working with biological data, writing careful and reproducible code, thinking critically about what your models do and do not tell you, and using AI as a collaborator rather than a substitute for your own judgment. Here you will put those skills together on a real, open scientific question: can you build a predictor of pathogenic variants in a gene whose variants cause a devastating pediatric disease, and can your predictor perform meaningfully alongside those of professional research groups? Your work on this problem is not a simulation. It is a small but real method for genome variant interpretation, in a field where that work matters.

1. Overview

You will build a machine-learning meta-predictor of missense variant pathogenicity, evaluate it on a self-test set of recently classified ClinVar variants, and apply it to the SNV-accessible missense variants of the ARSA gene. Your ARSA predictions will be assessed against experimental protein-stability measurements from the CAGI 7 ARSA challenge.

You may use any supervised learning method from scikit-learn, but at minimum you are required to train and compare two different methods, with at least some hyperparameter exploration for each.

2. Background: ARSA, metachromatic leukodystrophy, and the CAGI challenge

ARSA is arylsulfatase A, a lysosomal enzyme that hydrolyzes sulfatides. When ARSA is missing or dysfunctional, sulfatides accumulate, damage the myelin sheath, and cause metachromatic leukodystrophy (MLD). Disease severity varies with residual enzyme function: the late-infantile form is typically fatal by age five; juvenile and adult forms progress more slowly. Early identification of severe cases matters because the gene therapy Lenmeldy is effective only when administered before symptoms appear.

Experimental measurements of stability have been made for nearly every possible missense change in ARSA by the Gelb group at the University of Washington with a high-throughput assay. Stability is measured at 48 hours and reported as a continuous score; wildtype is approximately 0.784, and the clinically motivated threshold for predicting severe early-onset MLD is below 0.572. This dataset was the basis of the CAGI 7 ARSA challenge, and will be used to assess your model performance.

You are doing this project as an adjunct to the Critical Assessment of Genome Interpretation (CAGI), an international scientific experiment in which hundreds of researchers worldwide submit blind predictions on biological variants and have those predictions independently assessed against held-back experimental data. Your ARSA predictions will be evaluated in the same way, using the same experimental truth set and the same metrics.

Links:

Overall CAGI experiment: <https://genomeinterpretation.org/>

CAGI 7 ARSA challenge: <https://genomeinterpretation.org/cagi7-arsa.html>

Predecessor CAGI 6 ARSA challenge on enzymatic activity:
<https://genomeinterpretation.org/cagi6-lc-arsa.html>

For patient and family context, the MLD Foundation (a patient advocacy organization) maintains accessible information at <https://mldfoundation.org/>, including patient stories, research updates, and information on gene-therapy access.

3. Meta-predictors

The last three decades have produced an unusually large number of computational tools that score the likely impact of missense variants. The VIPdb catalogue (maintained at Berkeley) lists 407 variant-impact predictors as of VIPdb version 2.01 (August 2024). A revision is in preparation, curated with help from computational methods developed by a Berkeley undergraduate, that adds approximately 200 more predictors from the past two years. These predictors draw on different kinds of evidence: evolutionary conservation, protein structure, biophysical stability, deep-learning representations, prior pathogenicity annotations, population allele frequency. No single one captures everything.

A meta-predictor combines the outputs of several existing tools into a single prediction, typically by training a supervised model on ground-truth labels. Meta-predictors have consistently been among the best-performing methods in CAGI and similar assessments, because they combine partially non-overlapping perspectives on each variant: a conservation-based method, a structural method, and a deep-learning method will disagree on different variants, and a model that learns how to weight their outputs can do better than any one of them alone. The most widely used meta-predictor at present is REVEL, developed by Nilah Ioannidis (now on the faculty at UC Berkeley and UC Santa Cruz) and colleagues; it is a useful reference point for what a well-built meta-predictor looks like.

Your job is to build such a meta-predictor. For many well-built meta-predictors on suitable training data, cross-validation performance often exceeds that of the best individual component predictor. If yours does not, treat that as a diagnostic to investigate — often involving preprocessing, missingness handling, label balance, or cross-validation design — rather than as proof that the project has failed.

The component predictor scores you will use as features come pre-computed in dbNSFP, a database that aggregates per-variant annotations for every possible amino-acid-changing single-nucleotide variant in the human exome.

What pre-computed means is that the authors of dbNSFP have already run each component predictor on every such variant and stored the resulting scores, so you do not need to install, configure, or run the underlying tools yourself. This is what makes a project like this tractable on a course timescale: running SIFT, PolyPhen-2, CADD, AlphaMissense, and the others separately on tens of thousands of variants would take days and require installing each tool; reading the pre-computed scores from a single TSV takes seconds. dbNSFP collects scores from many widely used predictors (SIFT, PolyPhen-2, PROVEAN, FATHMM, VEST4, CADD, DANN, the Eigen scores, AlphaMissense, ESM-1b, and others), along with conservation scores (GERP++, phastCons, phyloP), gnomAD population allele frequencies, and structural and functional annotations. The course training, self-test, and ARSA prediction-target files are all derived from dbNSFP v4.9a (released August 2024).

dbNSFP only scores single-nucleotide variants (SNVs): missense substitutions reachable from the reference codon by a single base change. Some amino-acid substitutions, however, require two or three nucleotide changes within the same codon because no single base change produces the target amino acid under the standard codon table; these are multi-nucleotide variants (MNVs). Because dbNSFP does not cover MNVs, your predictable subset of ARSA is restricted to the roughly 2,000 SNV-accessible missense substitutions out of the 8,867 possible. Handling the MNV-only variants is a bonus opportunity (see section 16).

Pitfalls of circularity. Circular training is a pervasive issue for meta-predictors of pathogenicity, and it takes several forms you need to name in your report.

- **Direct ClinVar overlap.** Several of the component predictors in your feature set were themselves trained on variants drawn from ClinVar, which is also the source of your training labels. Some of those exact variants are likely to appear in your training data. This inflates your apparent cross-validation performance without reflecting true generalization.
- **Homologue overlap.** Even when the same variant is not in both training sets, variants at homologous positions in related proteins often are. A component predictor that saw a pathogenic variant at a conserved residue of a homologous sulfatase has, in effect, seen information about your ARSA target. Conservation-based and alignment-based predictors in particular carry this signal; you cannot easily strip it out.
- **Same-gene, different-variant overlap.** A component predictor trained on any ClinVar ARSA variants — even ones that are not in your training split — has implicit knowledge about the ARSA fitness landscape. Gene-level features (essentiality scores, pathway membership, and so on) compound this when they are computed using databases that include disease-gene annotations.
- **Data-augmentation cascade.** If you augment training with external sources (MaveDB, HGMD, more recent ClinVar releases, and so on), verify and report that the added variants are not part of any component predictor's training set. If they are, the circular-training problem compounds across both dimensions.

You do not have to solve these problems to pass this assignment. You do have to acknowledge them, describe how you tried to reduce their effect (or why you did not try), and factor them into your generalization discussion.

4. Permitted AI use

You are encouraged to use AI tools for essentially all of the coding and figure generation in this project. You must use the Berkeley-campus-licensed AI service; other consumer AI services (including Claude.ai, ChatGPT consumer accounts, and similar) are not permitted for this assignment.

You may not upload this assignment document, in whole or in part, to any AI tool. Write your own prompts describing the task and the constraints you need help with. The point of this project is that you design the analysis and the AI helps you carry it out, not the other way around.

AI-assisted work is expected for loading and cleaning the data, training models, running cross-validation, producing figures, writing the submission file, running the validator, and writing code comments. This is representative of how variant-effect prediction is actually done now in research settings.

The following parts of the report, however, must be written by you. In these sections, phrase-level AI editing — correcting a word, fixing a preposition, shortening a clause — is acceptable.

What is not acceptable is having AI review, rephrase, or restructure whole sentences or paragraphs, or asking AI to suggest how to structure an argument. Brainstorming with AI for these sections is also not permitted. We want to see your thinking and your sentences; for these items, AI should be your spellchecker, not your editor. Ask the course instructors if you are unsure what counts:

- Predict-your-own-performance.
- Ethical considerations.
- Limitations and self-assessment.
- Reflection on your use of AI.
- The honor-code statement.

If you genuinely did not use AI for some part of the project, that is fine; in your AI log, simply say so and briefly explain why. We are interested in honest accounts of how the work got done.

AI use is not a substitute for judgment. The code the AI writes is only as useful as your understanding of what it is supposed to do. If you cannot explain in your own words what a cell of your notebook does or why it is there, it will be hard for your grader to give you credit for it — and more importantly, it will be hard for you to trust your own work.

5. Learning objectives

This project exists so that you can learn, by doing, how real biological data analysis is done now. By the end of the project, we expect that you will be able to:

1. Apply supervised machine learning to a clinically consequential biological problem, using the same tools and workflow that professional research groups use.
2. Design and defend a disciplined train / cross-validate / test strategy, including how to notice and name data-leakage and circular-training issues that are easy to miss.
3. Build a meta-predictor, evaluate it effectively against its component predictors, and explain why it does or does not outperform them.
4. Use AI tools responsibly — to accelerate coding and figure generation, while retaining personal responsibility for design, interpretation, judgment, and written argument.
5. Connect computational predictions to biological mechanism (protein stability versus pathogenicity) and to clinical consequence (MLD severity, newborn screening, gene therapy access).
6. Recognize and articulate the ethical, social, and equity dimensions of computational variant interpretation in a real clinical context.
7. Produce a reproducible, well-documented analysis (notebook and report) that a peer or an assessor could evaluate and build on.

6. Grading

You will be graded on whether you built your meta-predictor correctly, evaluated it effectively, explained it clearly, and thought about what it means. We expect reasonable performance, and your predictor should be designed to generalize, but performance itself is not the primary consideration for your grade.

7. Honor code statement

Your report must end with the following statement, followed by your typed name and the date.

“As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I certify that the work submitted in this project is my own. I used only Berkeley-licensed AI services as permitted by this assignment, and acknowledged them in my AI log. I wrote the manual-only sections of this report myself, without AI drafting or substantive AI editing. I did not give or receive inappropriate help, and I have not shared my ARSA predictions with other students in this class. I have complied with the honor code and all stated course policies for this project.”

If any of the above is not true, do not include it in your report; instead, describe what happened in the reflection-on-AI-use section and talk to the course instructors before submitting.

8. Data files

The course will provide the following files. All data files are tab-separated values (TSV) — plain text files where columns are separated by tab characters and rows by newlines. In pandas they open with `pd.read_csv(path, sep='t')`. For each variant we provide features from a single primary transcript (the Ensembl canonical transcript); the specific transcript identifier for each variant will be noted in the file release. Full column descriptions are in Appendix A (variant-level) and Appendix B (gene-level) at the end of this document.

- **s26-c146-project-training.tsv** — approximately 13,000 missense variants across many genes, labeled via ClinVar as benign or pathogenic, with features from dbNSFP v4.9a (release date 8 August 2024). This is your training set.
- **s26-c146-project-test.tsv** — the self-test set: missense variants classified more recently in ClinVar than the training set, with the same feature columns. Use it only for final evaluation; do not train or tune on it. Released later in the project.
- **s26-c146-project-gene-features.tsv** — per-gene annotations joined by Ensembl gene ID (essentiality, pathway membership, function, GO terms, and similar). Some columns contain multiple values per gene (e.g., GO terms, pathway memberships) joined within the cell by semicolons; split with `.str.split(';')` if you need to work with them individually. These are not features for your model directly. They are available for an optional stratified analysis (see section 16).
- **s26-c146-project-cagi-7-arsa-sample.tsv** — 348 ARSA missense variants with experimental protein-stability scores from the CAGI 7 dataset alongside CAGI 6 enzymatic-activity measurements, with the same dbNSFP feature columns as the training and self-test files. This is the only ARSA-specific file with experimental measurements that you will see; the remainder of the CAGI 7 stability data is held back for the assessment of your ARSA predictions. Use this file for the descriptive analysis in project step 2; you may also fold it into training before generating ARSA predictions, see step 3.
- **s26-c146-arsa-cagi-snv-prediction.tsv** — a prediction template listing the approximately 2,000 SNV-accessible missense variants in ARSA after the signal peptide. You will replace placeholders with your predictions. Released later.
- **s26-c146-arsa-cagi-snv-validation.py** — a course-modified version of the CAGI 7 validation script. Run it on your prediction file before uploading.

Roles of the data files. The training set is for model selection and hyperparameter tuning by cross-validation. The self-test set is for one honest generalization estimate of pathogenicity-classification performance, used after your model and hyperparameters are frozen. The 348-variant ARSA sample is for the descriptive analysis in project step 2 and may optionally be

folded into training before you generate ARSA predictions. The ARSA prediction template is what you submit; your submission is scored against held-back CAGI 7 stability measurements that you do not see, using rank-based metrics (Kendall's τ and a high-specificity truncated AUC). In short, you do two distinct evaluations — self-test for pathogenicity classification on diverse genes, and the CAGI-style assessment for ARSA-specific stability ranking.

About the training labels. The binary pathogenicity labels in the training and self-test sets come from ClinVar, NCBI's public archive of human variant classifications. Classifications follow the ACMG/AMP guidelines (Richards et al., 2015; see Appendix D), under which a call of Pathogenic or Benign reflects approximately 99% confidence in the classification. Likely Pathogenic and Likely Benign reflect roughly 90% confidence; these are included in the training labels as pathogenic and benign respectively.

ClinVar is the most widely used source of variant-pathogenicity labels, but it is not perfect: submitter evidence quality varies, review status differs across records, and some classifications change over time as new evidence accumulates. Treat the labels as your best-available training signal, not as ground truth. The training labels also mix Pathogenic with Likely Pathogenic (and Benign with Likely Benign), so class-label confidence is not perfectly homogeneous — a small amount of label noise is expected.

A note on the submission format. The official CAGI 7 submission format requires a per-variant standard deviation. For this project you may enter a literal asterisk in the SD column; the course-modified validator accepts asterisks. Populating SD with meaningful values is a bonus. (For why the prediction target is limited to SNV-accessible variants, and what MNVs are, see section 3.)

9. Missingness

Missingness refers to feature values that are not present for some variants. In dbNSFP this is common for several reasons: some predictors only score residues within a high-confidence structural model; some require multi-species alignments and skip positions where alignment is poor; rare variants have no population allele frequency in gnomAD; and some predictors simply do not produce a score for every possible missense change. Most scikit-learn estimators will not accept NaN values directly.

You have several options for handling missingness, and different choices can meaningfully affect your model. Common approaches:

- **Drop.** Remove variants (or features) with any missing values. Simple; can discard a lot of informative data.
- **Simple imputation.** Fill missing values with the column mean, median, or a constant (zero, a sentinel like -1 , or the column-wide neutral value). Works well if missingness is roughly random; loses information if missingness is systematic.
- **Missingness-indicator features.** For each feature with missing values, add a binary column that records whether the value was missing. Lets your model learn that missingness itself is informative.
- **Algorithms that tolerate missingness.** Some tree ensembles and gradient-boosted methods accept NaN values natively and split on them; worth considering if you want to avoid imputation.

Describe in your report what you did about missingness and why. If a feature has very high missingness (say, more than a third of variants) your report should at least acknowledge that using it is risky.

10. Project steps

Do these in order. Each step should leave a clear trace in your notebook; the report summarizes and reflects.

- Step 1. **Explore the training data.** Check class balance. Quantify and describe missingness per column and per variant. Inspect how each component predictor score distributes across pathogenic and benign labels. Examine the correlation structure among predictors.
- Step 2. **Examine stability versus enzymatic activity.** Using s26-c146-project-cagi-7-arsa-sample.tsv, plot experimental ARSA protein stability against CAGI 6 enzymatic activity for the 348 variants. Describe what you see and what it implies for using a pathogenicity predictor to rank stability.
- Step 3. **Design your evaluation pipeline before you train anything.** Write it down in your notebook first. Default four phases: (i) cross-validate on the training set for method selection and hyperparameter tuning — do not touch the self-test set; (ii) freeze method and hyperparameters, train on the full training set, and evaluate once on the self-test set; (iii) optionally fold the 348-variant ARSA sample into training and retrain before generating predictions, reporting whether this changes anything visible on self-test — any such comparison is descriptive only and must not change your submitted pipeline; (iv) generate predictions and submit. Alternative pipelines are fine if justified. The non-negotiables: write it down first, and do not touch the self-test set during phase (i). Any external data must not overlap with any component predictor's training set.

Preprocessing inside cross-validation. Any step that learns from the data — imputation, scaling, feature selection — must be fit on training folds only, not on the full training set before CV. An sklearn Pipeline does this automatically. Fitting preprocessing outside the CV loop leaks information between folds and inflates your score. Use StratifiedKFold by default, given the class imbalance in ClinVar.

- Step 4. **Train at least two meta-predictors.** Compare two different scikit-learn classifiers (for example, random forest and k-NN). Run k-fold cross-validation on each. For each method, explore at least three or four values of at least two hyperparameters — by hand or with GridSearchCV. Report what you tried, what you chose, and why, with a short comment on hyperparameter sensitivity. A single default-settings model is not enough.
- Step 5. **Cross-validation caveat.** In your report, discuss whether your cross-validation performance is a reliable estimate of how your model will generalize. Consider how the component predictors in your feature set were themselves trained. You do not need to resolve the issue, but you must notice it.
- Step 6. **Evaluate on the self-test set.** Compute AUC and PR-AUC (both rank-based, no threshold needed) for your meta-predictor and for at least one component predictor from your feature set (a second is encouraged). Before comparing raw component predictors to your meta-predictor, align score directions so that larger values always mean more damaging — see section 11 and Appendix A for details. Discuss whether your meta-predictor outperforms its component(s) on each metric, and why. Threshold-based metrics (F1, MCC, balanced accuracy, confusion matrix) are welcome but not required.
- Step 7. **Train on the self-test set — a post-hoc diagnostic only.** Do this only after step 6 is locked in. It must not change your model, method, or pipeline. Run cross-validation on the self-test variants alone to produce a test-trained CV AUC: an estimate of the best performance achievable on the self-test distribution if you could train on it.

Report it alongside your training-CV AUC and your self-test AUC, and comment on what the three numbers tell you about generalization.

- Step 8. **Design a creative figure of your own.** Produce at least one figure that integrates multiple types of data and uses multiple modes of presentation in a single figure — not a panel of unrelated sub-plots. Examples: a heatmap with an overlaid scatter, or predictions alongside experimental measurements with annotated outliers. Caption it well; describe what you learned from it in your report. What counts as “creative” is up to you.
- Step 9. **Apply your model to ARSA.** Generate predictions for the ~2,000 SNV-accessible ARSA missense variants in the prediction template. Submit a continuous score for each variant — not a 0 / 1 label. You may submit your meta-predictor's pathogenicity probability directly; note in your methods that you are using it as a rank proxy for stability, a practical approximation rather than a claim that pathogenicity and stability are identical. Critical: higher submitted scores must mean more damaging / less stable. Check the orientation before submitting; the validator will not catch a conceptually inverted submission. (Mapping your output onto a calibrated stability scale is a distinction-tier extension; see section 16.)
- Step 10. **Validate the format of your submission.** Run `s26-c146-arsa-cagi-snv-validation.py` on your prediction file before uploading and confirm it reports the file as valid and complete.
- Step 11. **Predict your own performance.** Before you submit, write one or two sentences predicting the Kendall's τ your ARSA predictions will earn, with brief reasoning from your cross-validation and self-test numbers — what you expect to generalize, and what you do not. Put this in a clearly marked section of your report. You are graded on whether you made the prediction, not whether it turns out to be right.

11. Evaluation metrics

CAGI-style assessment of your ARSA predictions. Your ARSA predictions will be assessed against the CAGI 7 experimental stability measurements using two rank-based metrics: Kendall's τ (rank correlation) and a high-specificity truncated AUC at the severe early-onset MLD threshold of 0.572. Both metrics depend only on the ordering of your predictions, not on their absolute values, so you can submit your meta-predictor's pathogenicity probability directly — without mapping onto the stability scale — and still be scored.

1. **Kendall's τ rank correlation.** -1 to 1; higher is better. Measures how well your predictor orders variants from most to least damaging. Applied to continuous stability data in the CAGI assessment; we describe it here so you know what your ARSA submission will be scored on, not because we require you to compute it on self-test.
2. **High-specificity truncated AUC.** 0 to 1; higher is better. Rank-based within the high-specificity region. Focuses on predictor behavior at the clinically important operating point where false positives are costly.

Opt-in class leaderboard. A Google Sheet is available for the class. Students who opt in pick a handle and enter their numbers as they become available:
<https://docs.google.com/spreadsheets/d/1leCtkr2sGDVbDDgiAOFvlt6YUXXuVf1nnfjNieKRk8/edit?usp=sharing>.

Required self-test metrics. For your self-test evaluation (project step 6), compute two rank-based metrics for your meta-predictor and for at least one individual component predictor from your feature set: (a) area under the ROC curve (AUC), and (b) area under the precision-recall

curve (PR-AUC, also called average precision). Both are rank-based and require no threshold. They probe different aspects of ranking quality — AUC summarizes overall separation of pathogenic from benign across all thresholds, while PR-AUC is more sensitive to performance on the pathogenic minority class when the data are imbalanced. Reporting both gives you a more complete picture, and reasoning about cases where they disagree is itself good evidence of careful evaluation.

Align score directions before you compare. When computing AUC or PR-AUC for raw component predictors, first confirm that larger values consistently mean "more damaging." Several predictors in your feature set run the other way (SIFT, PROVEAN, FATHMM, ESM1b: smaller = more damaging) while others go up-is-damaging (PolyPhen-2, AlphaMissense, CADD). If you forget this step, your component-predictor AUCs will look inverted and your meta-predictor comparison will be wrong. Appendix A lists the direction of every score.

Self-test performance is not a forecast of your ARSA result. Your self-test AUC and PR-AUC measure pathogenicity classification across many genes. The ARSA assessment measures stability ranking on a single gene against a continuous experimental readout. They are related but not the same task. A strong self-test result is encouraging; it is not a direct prediction of your final ARSA Kendall's τ .

Optional additional metrics. If you want to pick a decision threshold on your meta-predictor and compute threshold-sensitive metrics (F1 score, Matthews correlation coefficient, balanced accuracy, a confusion matrix, or others), you are welcome to, and discussing these can strengthen your report. They are not required; rank-based metrics are sufficient for the core assignment.

12. Report

The report should read as a coherent scientific account of what you did, what you found, and what it means — not as a sequence of filled-in requirements. Length target: approximately four to seven pages at 11 pt, 1.5-line spacing, figures included. Submitted as PDF. This is a target, not a strict cap; concise clarity matters more than hitting a specific length. Your notebook is the primary place for all code and exploratory work. The report is a more polished narrative. The sections below are required and should appear in this order. You are allowed and encouraged to go beyond these minima; what is listed here is the baseline.

Introduction and goals

About half a page. One paragraph motivating the problem in your own words (what ARSA is, why its variants matter, what your meta-predictor does). One paragraph stating the specific goals of your particular project.

Data and methods

About three quarters of a page, with the content broken out explicitly. Write this as prose, but make sure each of the following items is present and identifiable:

- Which features from the training set you used, and any that you deliberately excluded (including any features you built or derived).
- How you handled missingness, and a one-sentence rationale.
- Which two (or more) ML methods you used, and why you chose them.
- Your train / cross-validate / test strategy, including whether and when you folded the CAGI 7 sample into training.

- What hyperparameters you explored, and how you settled on your final settings.
- Your cross-validation caveat discussion (see project step 5): is your CV AUC a reliable estimate of generalization, and why or why not?

Predict your own performance

One or two sentences, written before you see the final assessment results: your predicted Kendall's τ on the CAGI ARSA dataset, with brief reasoning. Manual-only.

Results

About one and a half pages, built around the following figures and tables. You may include additional figures and tables in the report freely; you may include further exploratory figures in the notebook.

- **Stability versus enzymatic activity.** A scatter plot of the 348 CAGI 7 sample variants, plus a short paragraph interpreting what you see and what it implies for using a pathogenicity predictor to rank stability.
- **Training-set score distributions.** A violin or overlaid KDE plot showing how at least three component predictor scores distribute across benign and pathogenic training variants.
- **Self-test ROC.** ROC curves for your meta-predictor alongside at least one individual component predictor (two is encouraged), on the self-test set.
- **Self-test metric table.** A small table in the report listing AUC and PR-AUC (average precision) on the self-test set for your meta-predictor and for each component predictor you compared against. This table must appear in the report itself, not only in the notebook.
- **Feature importance.** Feature importances from your random forest, or an equivalent interpretability plot if you used a non-tree method as your primary model.
- **Generalization report.** The three numbers from project step 7 — cross-validation AUC, self-test AUC, and test-trained cross-validation AUC — with a short paragraph interpreting the differences among them.
- **Your own creative figure.** At least one additional figure of your own design that integrates multiple types of data and multiple modes of presentation into one figure (see project step 8). Caption it well and write two or three sentences in the report describing what you learned from making and looking at it.
- **ARSA predictions.** A histogram of your predicted scores for the ARSA prediction set, plus one short paragraph summarizing what your predictions look like overall.

Ethical considerations

About one page, manual. Pick at least three of the prompts below and engage with them substantively. You are welcome and encouraged to write about others, to combine prompts, or to introduce considerations not on this list. You are not being asked to take sides; you are being asked to notice considerations honestly, articulate them clearly, and say what they imply.

- Population bias in training data.** ClinVar and gnomAD are heavily biased toward people of European ancestry. How might this bias affect the performance of your meta-predictor for patients from underrepresented populations, and what are the clinical consequences?
- Equity of access to treatment.** Lenmeldy lists for approximately US\$4.25 million per patient. In what countries and for which families is it actually available? What role, if any, should computational prediction play if only a small fraction of identified patients can actually be treated?

- (c) **Reliability thresholds.** At what level of accuracy does a variant-effect predictor become usable for clinical decisions? Who should decide where the line is drawn, and how should uncertainty in the prediction itself factor in?
- (d) **Sequencing versus biochemical screening.** Current MLD newborn screening uses biochemical and enzymatic assays. Adding sequencing, or replacing enzymatic screening with sequencing informed by computational prediction, has different ethical and practical implications. Consider cost, coverage, equity, and interpretive burden.
- (e) **Required versus voluntary newborn screening.** Newborn screening in most jurisdictions is mandatory or quasi-mandatory, with limited parental consent. Does expanding screening to include predicted-stability-based calls change the ethical calculus?
- (f) **Geographic variation in newborn screening and treatment.** MLD appears on the recommended newborn-screening panel in some US states and countries but not others. What obligations, if any, follow from the existence of a life-saving, pre-symptomatic-only intervention, and how do those obligations differ across jurisdictions?
- (g) **Dried blood spots and genetic privacy.** California has archived the dried blood spots of every baby born in the state since 1982, unless the family has specifically requested destruction. Students born in California will themselves have dried blood spots on file unless they had them destroyed. What does it mean that the state has access to this genetic material, and what changes if predictions like yours can now be made from it?
- (h) **Asymmetry of prediction errors.** A missed severe variant means a family loses the treatment window. A false severe call anchors a family on a terrible diagnosis and can lead to unnecessary interventions. How should a predictor be designed or deployed given this asymmetry?
- (i) **Pseudodeficiency variants and over-diagnosis.** Some ARSA variants reduce activity to 10–15 % of wildtype without causing disease. How should a prediction system handle these cases, and what does it imply about the reliability of any single molecular metric?
- (j) **Dual use.** Variant-effect predictors can also be used in prenatal or pre-implantation genetic testing. Does your project work equally well for those uses, and do you endorse that use?
- (k) **Accountability.** When a predictor informs a treatment decision and the decision turns out to be wrong, who is responsible — the model's authors, the clinician, the testing lab, the screening program, the vendor?
- (l) **Clinical trial participation and representation.** The pivotal trials for MLD gene therapy recruited from specific sites and populations. What does that mean for the generalizability of efficacy estimates, and for which families will predictive tools actually help?
- (m) **Privacy and publicity for severely ill patients.** Photographs of children with severe MLD are frequently used in fundraising, research communication, and even lectures in courses like this one. What should the default be — showing, de-identifying, or omitting — and who should decide?
- (n) **Data provenance and consent for experimental data.** The DMS data you are training against came from specific patient samples, research cell lines, and investigator time. What obligations do downstream modelers have to the people and programs that generated the data?
- (o) **Open science and reproducibility.** Your predictions will be evaluated by an independent assessor. What would you need to publish, and in what form, for someone else to reproduce your work exactly?

Limitations and self-assessment

Short, manual. What might be wrong with your model, where you expect it to fail on unseen variants, what you would investigate next. At minimum, include one to three sentences acknowledging that you are using a pathogenicity-trained model to rank protein stability: destabilization is only one route to pathogenicity (catalysis, metal coordination, multimerization, lysosomal trafficking, and splicing are others), so a pathogenicity-trained model may systematically mis-rank stable-but-functionally-disrupting variants. This brief acknowledgment is required; the fuller analysis is an option in section 16.

Reflection on your use of AI

About half a page, manual. Describe at least two concrete moments where the AI did something you did not expect — where it struggled with a problem in a way that surprised you, made a choice you disagreed with, appeared to understand a concept and then did not, or produced something that looked right but was subtly wrong. Say what you did about it and what you learned. We are looking for genuine friction. “The AI wrote code and it worked” is not a useful reflection.

Honor code statement

Copy the honor-code statement from section 7 verbatim, then type your name and the date.

13. Submission

Submit five files to Gradescope by 10:00 pm on Friday 1 May 2026:

1. Your notebook (.ipynb).
2. A PDF export of your notebook.
3. Your ARSA prediction file, named `s26-c146-arsa-cagi-snv-prediction-lastname-firstname.tsv` (replacing lastname and firstname with your own, lowercase, hyphen-separated).
4. Your report as a PDF.
5. Your AI log as a single concatenated PDF covering every AI session you used during the project, in the order they occurred.

Before submitting, work through the checklist at the end of section 14, and run the course validator on your prediction file to confirm it reports the file as valid and complete:

```
python s26-c146-arsa-cagi-snv-validation.py s26-c146-arsa-cagi-snv-prediction-  
lastname-firstname.tsv s26-c146-arsa-cagi-snv-prediction.tsv
```

14. Helpful hints for success

Practical hints that tend to make projects like this go well.

As you start

- **Lock your train / validate / test strategy before you train anything.** Write your plan down first. Don't change it after you see performance numbers.
- **Do not touch the self-test set until your model is frozen.** The self-test is your honest generalization estimate; every time you peek at it while tuning, you burn a little of that honesty.

- **Keep a running notebook commentary.** Write down what you tried, what worked, what didn't, and what you were thinking. Future-you and your grader will both thank you.
- **Validate your submission file before you polish your report.** Running the validator takes seconds; discovering a format problem at 9:55 pm on the due date is another story.
- **Start with a defensible baseline, then improve.** A clean random forest or regularized logistic regression with median imputation and missingness indicators, using stratified k-fold cross-validation, is a perfectly good starting point. Improvements on that baseline are what you report.
- **Reproducibility is a courtesy to your future self.** Set and report your random seeds. Note the versions of your core packages (scikit-learn, pandas, numpy) at the top of your notebook. A result that cannot be rerun is not a result.
- **Depth over breadth.** In section 16, one well-executed extension is generally more valuable than several shallow ones. Pick something you can investigate carefully rather than trying to touch everything.
- **Don't chase leaderboard noise.** Your self-test AUC is an estimate with uncertainty. Small improvements that happen to show up on self-test may not survive on the CAGI-style assessment.
- **Ask for help early.** Lab sessions and office hours both exist for this; bring questions to either. If you are stuck on something that you think is 'basic,' ask anyway — most of the time the basic question has a non-basic answer.
- **Aim for a readable notebook and a clean report.** The project rewards clarity as much as cleverness.

Ideas for how to approach this work

This project exists so you can learn to do data science and biology together, and do both with integrity. Before the mechanics, it is worth stating what we actually value, because those values — more than the numbers you produce — are what we most want to teach. Keep this section in mind as you work; if you internalize these principles, the rest of the assignment follows naturally.

Honest evaluation matters more than a high score. A model that scores slightly lower under scrupulous evaluation is scientifically better than one that scores higher through leakage, circular training, or post-hoc tuning. Leaderboard position is not what we grade. The moment you find yourself tempted to peek at the self-test set during tuning, or to use a diagnostic number to change your model, is the moment self-deception begins. Recognize it, name it, and stop.

Labels are signal, not ground truth. ClinVar is our best available source of pathogenicity labels, but it is a human-curated resource with variable evidence quality. Likely Pathogenic and Likely Benign are lower-confidence calls than Pathogenic and Benign; some labels will be revised over time. Every label in science has a provenance, an uncertainty, and a scope. Ask those questions of any dataset you work with.

Know what your data mean. Your features come from dbNSFP, which precomputes scores from specific tools. Your labels come from ClinVar under the ACMG/AMP framework. Your ARSA target comes from a specific stability assay at 48 hours. Each of those choices carries assumptions. Understand what each measurement captures, what it misses, and why values can be absent.

Preprocessing is part of the model. Imputation, scaling, missingness indicators, score-direction alignment, feature selection — these are modeling decisions that encode assumptions

about your data. Keep them inside your cross-validation loop, document them, and treat them as part of your scientific reasoning.

Simplicity first, then justified complexity. A clean, honest baseline that you understand is worth more than an elaborate pipeline you do not. Start simple. Add complexity only when evidence supports it.

Metrics are tools, not verdicts. No single metric is universally best. AUC, PR-AUC, Kendall's τ , and a high-specificity truncated AUC all answer different questions. Ask what kind of mistake each metric cares about and whether that mistake matters biologically or clinically.

Mechanism matters. A statistically accurate model can still be biologically wrong. Pathogenicity is not identical to destabilization: a stable variant can still disrupt catalysis, metal coordination, multimerization, trafficking, or splicing. Predict where your model is likely to be wrong in mechanistic terms, not just by saying the AUC is imperfect.

Study your failures. What distinguishes good scientists is what they learn from cases their model gets wrong. Look at specific variants your predictor misranks and ask why.

Calibrate your claims. The right conclusion is often conditional and modest. "This predictor appears to rank pathogenicity reasonably on a held-out ClinVar set and may provide useful ARSA prioritization, though its stability ranking is an approximation likely to fail in specific mechanistic regimes" is a better answer than anything more sweeping.

Reproducibility is part of honesty. A result that cannot be reconstructed is not a scientific result. Set and report your random seeds, note your package versions, and write a notebook someone else could rerun.

AI accelerates; judgment is yours. AI can generate code and draft prose faster than you can. It cannot detect leakage, notice a conceptual mismatch, or tell you that a beautiful figure is saying something false. Those are the hardest and most valuable parts of the work, and they are yours.

Ethical reflection is part of method. Who benefits, who is left out, and what a false call would cost are scientific questions, not afterwords. Treat them that way.

One closing note. Self-deception is the greatest risk a scientist faces, and it is pervasive — no one is immune, and the most confident-sounding results often have the most room for it. It is precisely why independent community assessments like CAGI exist: blind evaluation against held-back experimental data keeps the field honest about what methods can actually do. Successful scientists do not escape this risk; they do their best to notice it, name it, and work against it in themselves. Do the same.

One last observation. Self-deception is among the greatest risks to a scientist, and it is pervasive. It is precisely why independent assessments like CAGI exist — to test predictions against evidence that the predictor's authors have not seen. Successful scientists know this and work, constantly, to avoid being self-deluded: they build habits of honest evaluation, guard against looking at data they have promised not to look at, and calibrate their claims carefully. Those habits are what this project is designed to cultivate.

Before you submit — a checklist

A short self-check before submission.

1. Targets identified. The training target (ClinVar pathogenicity class) and the final scientific target (ARSA stability ranking) are stated in your own words, along with how they are and are not aligned.

- 2. Pipeline locked first.** The train/validate/test pipeline was written down before any model was trained, and that record is present in the notebook.
- 3. Preprocessing inside cross-validation.** Imputation, scaling, and any other data-learned step were fit on training folds only, via an sklearn Pipeline or equivalent.
- 4. Stratified cross-validation used.** Stratified k-fold was the default, or an alternative is justified in the report.
- 5. Score directions aligned.** When comparing raw component predictors to the meta-predictor, larger values mean "more damaging" for every predictor compared.
- 6. One frozen self-test.** The self-test set was evaluated exactly once, after the model and hyperparameters were frozen. AUC and PR-AUC for the meta-predictor and at least one component predictor appear in a table in the report.
- 7. Post-hoc diagnostics did not feed back.** The test-trained CV experiment was run only after the self-test evaluation was locked in, and did not change the model.
- 8. Submission orientation checked.** Higher submitted scores correspond to more-damaging / less-stable variants; the orientation was verified by spot-checking known-pathogenic and known-benign variants.
- 9. Failure modes identified.** At least one biologically plausible class of variants where the model is likely systematically wrong is named in the report — not only that "AUC is imperfect."
- 10. Claims calibrated.** Conclusions are stated conditionally and scoped; no claim exceeds what the evidence supports.
- 11. Reproducible.** Random seeds are set and reported; package versions are noted; the notebook runs end-to-end from a fresh kernel.
- 12. AI log and honor code.** The AI log is present and honest; the manual-only sections were written by the student; the honor code appears with typed name and the date.

15. Accessibility and access

We intend for everyone in class to be able to succeed at this project, including students with documented disabilities, different educational backgrounds, different ways of processing information, different health or life circumstances, different language backgrounds, and different relationships to the emotional content of the material. If any part of this assignment creates a barrier for you, or if an adjustment would let you engage more fully, please tell the course instructors as early as you can. You do not need a formal accommodation letter to start a conversation, and please do not share medical information with course instructors. Students registered with the Disabled Students' Program should send their letters to the course instructors if they have not already done so.

16. Distinction, bonus, and extra credit

There are many ways to take this project beyond the core requirements. Any of the items below can earn distinction, bonus credit, or extra credit depending on the ambition and quality of the work you put into it. Pick one or more, or propose your own and run it by the course instructors. Items may be done as a group; if so, clearly indicate which components were group work.

1. **Stability versus pathogenicity analysis.** Your training labels are binary pathogenicity calls; the CAGI target is a continuous stability score. Using a pathogenicity predictor to rank stability is defensible because destabilized proteins are preferentially degraded and

there is quantitative evidence linking destabilization to MLD severity; but the mapping is asymmetric. A stable variant can still be pathogenic if it disrupts catalysis, metal coordination, multimerization, lysosomal trafficking, or splicing. A well-known example is ARSA A214V, which folds normally but has near-zero enzymatic activity and still causes severe MLD. Analyze this: (i) explain the direction and quantitative nature of the stability-to-pathogenicity link in your own words, (ii) identify at least two regions or residue classes in ARSA where you expect your model to be systematically wrong as a stability predictor, and (iii) propose and optionally implement a better mapping from your pathogenicity output onto a calibrated stability scale.

2. **Gene-level stratification analysis.** Using the gene-level features file, ask whether your model works equally well across different kinds of genes. Pick one or more stratifications (e.g., essentiality, pathway membership, a chosen GO term, or the number of ClinVar entries per gene), compute self-test AUC within each group, and interpret the differences. What do they suggest about where your meta-predictor is likely to generalize well and where not?
3. **Circular-training quantification.** Design and run an experiment that gives quantitative evidence for how much your cross-validation AUC overstates your true generalization. Split variants by date of ClinVar submission, or by publication year of the component predictors, and show how performance varies with the age of the held-out variants.
4. **Direct stability regression.** Build a regression model trained directly on a continuous stability or activity target (for example, on the 348-variant CAGI 7 sample file or on an analogous MaveDB dataset). Compare to your classification baseline.
5. **MNV-only ARSA missense variants.** Only about 2,000 of the 8,867 ARSA missense variants are reachable by a single-nucleotide change; dbNSFP does not score the others. Using features of your choice (protein language models, structural features, MaveDB data, per-residue scores), generate stability predictions for the remaining MNV-only variants. We cannot assess these through the course pipeline, but describe your approach, show any self-evaluation you can do, and submit the predictions as a second file.
6. **Structural and mechanistic features.** Using the ARSA crystal structure (PDB 1AUK or 1N2L), add features for distance to the catalytic Cys69 / formylglycine active site, proximity to the Mg^{2+} -coordination residue at N284, solvent accessibility, or multimerization interface residues. Show whether these features improve the model specifically on variants your baseline gets wrong.
7. **Protein language model features.** Extract representations from ESM-2 (or an equivalent publicly available protein language model beyond the ESM-1b score already in your features) and use them as additional input features.
8. **MaveDB augmentation.** Incorporate quantitative variant-effect measurements from MaveDB deep-mutational-scanning experiments beyond ARSA. Verify and report that the added variants are not part of the training set of any component predictor you use.
9. **Stacking.** Build a stacked ensemble of two or more different meta-predictors (random forest + gradient boosting + logistic regression, say), and compare to each base model.
10. **Calibration.** Examine and report on the calibration of your predictor's probabilities (reliability diagram, Brier score). Apply Platt scaling or isotonic regression if appropriate.
11. **Uncertainty quantification.** Use the spread across your ensemble members or cross-validation folds to estimate meaningful per-variant standard deviations, and populate the SD column of your submission with real values. Describe the method.

12. **Alternative cross-validation schemes.** Compare random k-fold against gene-grouped cross-validation (each gene entirely in one fold). Report how much the performance differs and what that tells you.
13. **Systematic hyperparameter search.** Go beyond the minimum exploration required in project step 4: use grid search, random search, or a Bayesian approach, with honest separation from the self-test set.
14. **Feature selection.** Run a principled feature-selection procedure (recursive feature elimination, permutation importance, embedded regularization). Comment on which features survive and why.
15. **Class-imbalance handling.** ClinVar is imbalanced. Compare approaches (resampling, class weighting, threshold tuning, calibrated probability cutoffs) and report what actually helps.
16. **Residue-level analysis on ARSA.** Aggregate your ARSA predictions per residue and compare to the known structural and functional annotation of ARSA. Where are your predictions most uncertain, and does that line up with specific biochemical features?
17. **Clinician-oriented analysis.** Add to your report a short paragraph framed for a clinical audience: what your predictor says about specific variants likely to be of clinical interest, and what caveats a clinician should keep in mind when using it.
18. **Your own idea.** If you see something interesting in the data or think of an approach we did not list, pursue it. Describe clearly what you did and why, and check in with course instructors if you want feedback before committing significant time.

17. Alternative projects

If you want to pursue a different data science for biology project, you may do so, but only with prior written approval from the course instructors confirming that your alternative is feasible and sufficiently challenging. Coordinate with us no later than Friday 24 April. All students, regardless of which option they choose, are strongly encouraged to go beyond the basic project.

18. Acknowledgments

This project exists because of the work of many people. We gratefully acknowledge the following contributions:

- **This assignment** was developed by graduate student instructor Jeremy Marcus and supervised by Steven E. Brenner. It was created with the assistance of AI.
- **The ARSA project concept** derives from the CAGI MLD Boot Camp (<https://genomeinterpretation.org/bootcamp-2022.html>), whose organizers were Courtney Astore, Constantina Bakolitsa, Steven Brenner, Wyatt Clark, Shantanu Jain, Akash Kamandula, Reet Mishra, Vikas Pejaver, Predrag Radivojac, Rashika Ramola, Marena Trinidad, Michelle Velyunskiy, and Daniel Zeiberg. We gratefully acknowledge their work, and the work of the bootcamp participants whose results showed that carefully trained, modestly resourced models can compete with much more elaborate ones on this kind of task.
- **The CAGI 6 ARSA enzymatic activity challenge** and its assessment (Jain et al., 2025; see Appendix D) established the predecessor task on which the clinical severity thresholds are based. We acknowledge the challenge organizers, the predicting teams, and the assessors.

- **The CAGI 7 ARSA protein stability challenge** provided the experimental target against which your predictions will be assessed. The stability dataset was generated by the Gelb group at the University of Washington. Challenge materials and the independent assessment are the work of the CAGI organizers and of the CAGI 7 ARSA assessor, Yu-Jen (Jennifer) Lin.
- **dbNSFP v4.9a** (release date 8 August 2024) is the source of all your component predictor scores and conservation features. We acknowledge Xiaoming Liu and colleagues.

Remember: the work you are doing in this project is the kind of work a graduate student or scientist might undertake. Using tools that did not exist even last semester, you are creating a new method in two weeks. Whatever your Kendall's τ turns out to be, you have built something real. Biology is a data science, and you are using the most modern tools and concepts to engage with it. We hope you finish this course proud of what you have learned and curious about what comes next.

Appendix A. Variant-level column descriptions

The training set and self-test set share the same columns. Direction indicates whether larger values correspond to more damaging / more pathogenic (↑ damaging) or to less damaging (↓ damaging). Many columns will have missing values for some variants; section 9 describes how to handle missingness.

Variant coordinates and gene identifier:

Column	Description	Direction
chr	Chromosome.	—
pos	Genomic position (1-based).	—
ref	Reference allele.	—
alt	Alternative allele.	—
Ensembl_geneid	Ensembl gene ID. Use this to join to the gene-level file.	—

Allele frequency:

Column	Description	Direction
gnomAD_exomes_AF	Alternative allele frequency in gnomAD v4.0.0 exomes. Common variants are unlikely to cause rare Mendelian disease.	—

Conservation scores (more conserved sites are more likely to be functionally important):

Column	Description	Direction
GERP++_RS	GERP++ rejected-substitutions score. Approximately -12.3 to 6.17.	↑ conserved
phastCons100way Vertebrate	phastCons across 100 vertebrate genomes. 0 to 1.	↑ conserved
phastCons470way_mammalian	phastCons across 470 mammalian genomes. 0 to 1.	↑ conserved
phyloP100way Vertebrate	phyloP across 100 vertebrate genomes. Approximately -20 to 10.	↑ conserved
phyloP470way_mammalian	phyloP across 470 mammalian genomes. Approximately -20 to 12.	↑ conserved

Component variant-impact predictor scores. These are your main features. Directions differ across predictors; the note below describes the meta-predictor exclusions.

Column	Description	Direction
SIFT_score	Sequence-based prediction from evolutionary information. 0 to 1.	↓ damaging
Polyphen2_HVAR_score	PolyPhen-2 HumVar score combining sequence and structural features. 0 to 1.	↑ damaging
PROVEAN_score	PROVEAN score. Approximately -14 to 14.	↓ damaging
FATHMM_score	FATHMM weighted score, trained with Disease Ontology. Approximately -16 to 11.	↓ damaging
VEST4_score	VEST 4.0 score (meta-predictor / ensemble method). 0 to 1.	↑ damaging

Column	Description	Direction
CADD_raw	CADD raw score (ensemble of many annotations). Approximately -28 to 26.	↑ damaging
DANN_score	DANN score (deep neural network trained on CADD's training data). 0 to 1.	↑ damaging
Eigen-raw_coding	Eigen score for coding SNVs; unsupervised aggregate of conservation, allele frequency, and deleteriousness.	↑ damaging
Eigen-PC-raw_coding	Eigen-PC score (genome-wide, unsupervised ensemble).	↑ damaging
AlphaMissense_score	AlphaMissense, a supervised end-to-end deep-learning pathogenicity score. 0 to 1.	↑ damaging
ESM1b_score	ESM-1b log-likelihood-ratio score from a 650 M-parameter protein language model. Approximately -24 to 7.	↓ damaging

Note on meta-predictors in the feature set. MetaSVM, MetaLR, REVEL, EVE, and MutPred — ensemble or deep-learning predictors trained on ClinVar-like or homologous-sequence data — have been excluded from your feature set, because including them would give your meta-predictor an unfair shortcut. Several remaining features (notably VEST4, CADD, DANN, and the Eigen scores) are themselves meta-predictors or ensemble methods. You may use them, but your report should note that their training data likely overlaps with your training variants, which contributes to the circular-training problem discussed in section 3.

Labels:

Column	Description	Direction
Clinical_significance	ClinVar clinical significance as a text string (e.g., Pathogenic, Benign, Likely pathogenic, Likely benign).	—
Y	Binary label (0 = benign or likely benign; 1 = pathogenic or likely pathogenic). This is your training target.	—

Appendix B. Gene-level column descriptions

One row per gene; joined to the variant-level files by `Ensembl_geneid`. These fields are text or semicolon-delimited categorical annotations; use them for stratified analyses (see section 16), not as direct model features.

Column	Description
<code>Ensembl_geneid</code>	Ensembl gene ID (join key).
<code>Pathway(ConsensusPathDB)</code>	Pathway membership from ConsensusPathDB.
<code>Pathway(KEGG)_full</code>	Pathway membership from KEGG.
<code>Function_description</code>	UniProt function description.
<code>MIM_phenotype_id</code>	OMIM phenotype identifiers.
<code>Trait_association(GWAS)</code>	Reported GWAS trait associations.
<code>GO_biological_process</code>	Gene Ontology biological-process annotations.
<code>GO_cellular_component</code>	Gene Ontology cellular-component annotations.
<code>GO_molecular_function</code>	Gene Ontology molecular-function annotations.

Appendix C. Suggested notebook section headers

Copy the following as markdown headers into your notebook, in this order, and fill in the code and commentary as you work. You do not have to match these exactly, but a structure like this will make your notebook easier to grade.

1. Setup and imports
2. Load training data
3. Exploratory data analysis of the training set (class balance, missingness, score distributions, correlation)
4. Enzymatic activity vs. stability, using the 348 CAGI 7 ARSA sample variants
5. Train / validate / test strategy
6. Model 1 training, hyperparameter exploration, and cross-validation
7. Model 2 training, hyperparameter exploration, and cross-validation
8. Cross-validation summary and caveat discussion
9. Self-test evaluation (ROC curves, precision-recall curves, AUC and PR-AUC)
10. Self-test metric table
11. Test-trained CV experiment
12. Your creative figure
13. Load the ARSA prediction template
14. Generate ARSA predictions with your chosen model
15. ARSA prediction-score histogram
16. Write and validate the submission file
17. Predict-your-own-performance note

Appendix D. Optional reading to learn more

These are optional references for students who want to go deeper on the biology, the methods, or the CAGI experiment itself. Each is annotated with what it covers and why it might be worth your time. Click any DOI to open the article.

On variant classification standards

- Richards S, Aziz N, Bale S, Bick D, Das S, Gastier-Foster J, et al. Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine*. 2015;17(5):405–424. doi:[10.1038/gim.2015.30](https://doi.org/10.1038/gim.2015.30). PMID:[25741868](https://pubmed.ncbi.nlm.nih.gov/25741868/). PMCID:[PMC4544753](https://pubmed.ncbi.nlm.nih.gov/PMC4544753/)

The ACMG/AMP guidelines paper. This is the framework under which ClinVar variants in your training and self-test sets are classified. A variant called Pathogenic or Benign under these guidelines represents approximately 99% confidence; Likely Pathogenic and Likely Benign represent roughly 90% confidence. Useful if you want to understand what the training labels actually mean and what kinds of evidence go into each call.

On ARSA biology and metachromatic leukodystrophy

- Gieselmann V. Metachromatic leukodystrophy: genetics, pathogenesis and therapeutic options. *Acta Paediatrica*. 2008;97(s457):15–21. doi:[10.1111/j.1651-2227.2008.00648.x](https://doi.org/10.1111/j.1651-2227.2008.00648.x). PMID:[18339182](https://pubmed.ncbi.nlm.nih.gov/18339182/)

A concise clinical-plus-molecular review, and one of the most widely cited introductions to MLD. Good grounding in how ARSA deficiency produces disease and what therapeutic strategies have been tried.

- Cesani M, Liorioli L, Grossi S, Amico G, Fumagalli F, Spiga I, et al. Mutation update of ARSA and PSAP genes causing metachromatic leukodystrophy. *Human Mutation*. 2016;37(1):16–27. doi:[10.1002/humu.22919](https://doi.org/10.1002/humu.22919). PMID:[26462614](https://pubmed.ncbi.nlm.nih.gov/26462614/)

A systematic catalogue of pathogenic ARSA and PSAP variants with associated phenotypes. Useful if you want to cross-reference your model's predictions against previously reported disease-associated variants, or if you're thinking about the stability-versus-pathogenicity distinction for a specific residue.

On meta-predictor design

- Kircher M, Witten DM, Jain P, O’Roak BJ, Cooper GM, Shendure J. A general framework for estimating the relative pathogenicity of human genetic variants. *Nature Genetics*. 2014;46(3):310–315. doi:[10.1038/ng.2892](https://doi.org/10.1038/ng.2892). PMID:[24487276](https://pubmed.ncbi.nlm.nih.gov/24487276/). PMCID:[PMC3992975](https://pubmed.ncbi.nlm.nih.gov/PMC3992975/)

The CADD paper. CADD is an earlier and simpler meta-predictor than REVEL — a linear-ish combination of many annotations, trained on simulated-vs-observed variation — and it is a good reference point for the basic architecture of an ensemble pathogenicity score.

- Ioannidis NM, Rothstein JH, Pejaver V, Middha S, McDonnell SK, Baheti S, et al. REVEL: An ensemble method for predicting the pathogenicity of rare missense variants. *American Journal of Human Genetics*. 2016;99(4):877–885. doi:[10.1016/j.ajhg.2016.08.016](https://doi.org/10.1016/j.ajhg.2016.08.016). PMID:[27666373](https://pubmed.ncbi.nlm.nih.gov/27666373/). PMCID:[PMC5065685](https://pubmed.ncbi.nlm.nih.gov/PMC5065685/)

The canonical modern reference for meta-predictor construction. REVEL combines thirteen component predictors with a random forest and is the template many later meta-predictors follow. Short, well-written, and directly relevant to your project.

- Lin YJ, Menon AS, Hu Z, Brenner SE. Variant Impact Predictor database (VIPdb), version 2: trends from three decades of genetic variant impact predictors. *Human Genomics*. 2024;18(1):90. doi:[10.1186/s40246-024-00663-z](https://doi.org/10.1186/s40246-024-00663-z). PMID:[39198917](https://pubmed.ncbi.nlm.nih.gov/39198917/). PMCID:[PMC11360829](https://pubmed.ncbi.nlm.nih.gov/PMC11360829/)

The VIPdb paper referenced in section 3. Useful if you want to get a sense of the overall landscape of variant-impact predictors — which kinds of methods exist, which inputs they use, which have been assessed and with what results, and how the field has grown.

On the CAGI experiment and the ARSA challenges

- Jain S, Trinidad M, Nguyen TB, Jones K, Diaz Neto S, Ge F, et al. Evaluation of enzyme activity predictions for variants of unknown significance in Arylsulfatase A. *Human Genetics*. 2025;144(2–3):295–308. doi:[10.1007/s00439-025-02731-3](https://doi.org/10.1007/s00439-025-02731-3). PMID:[40055237](https://pubmed.ncbi.nlm.nih.gov/40055237/). PMCID:[PMC12122056](https://pubmed.ncbi.nlm.nih.gov/PMC12122056/)

The CAGI 6 ARSA assessment paper, the immediate predecessor to the challenge you are working on. It describes how the patient phenotype matrix was built, how the clinical severity thresholds (0.572, 0.659, 0.729, 0.784) were derived, and how the CAGI 6 participating teams performed. The CAGI MLD Boot Camp — a separate training event whose participant models did well against more elaborate professional submissions — was held in 2022 (<https://genomeinterpretation.org/bootcamp-2022.html>) and inspired the structure of this course project.

- Critical Assessment of Genome Interpretation Consortium. CAGI, the Critical Assessment of Genome Interpretation, establishes progress and prospects for computational genetic variant interpretation methods. *Genome Biology*. 2024;25(1):53. doi:[10.1186/s13059-023-03113-6](https://doi.org/10.1186/s13059-023-03113-6). PMID:[38389099](https://pubmed.ncbi.nlm.nih.gov/38389099/). PMCID:[PMC10882881](https://pubmed.ncbi.nlm.nih.gov/PMC10882881/)

A retrospective on CAGI history, scope, and lessons across all its challenges to date. Useful for broader context on community assessment experiments — what they have shown, and what they have been unable to show.

Web links:

Overall CAGI experiment: <https://genomeinterpretation.org/>

CAGI 7 ARSA challenge: <https://genomeinterpretation.org/cagi7-arsa.html>

CAGI 6 ARSA challenge: <https://genomeinterpretation.org/cagi6-lc-arsa.html>

CAGI MLD Boot Camp: <https://genomeinterpretation.org/bootcamp-2022.html>

MLD Foundation (patient advocacy): <https://mldfoundation.org/>